Acme Corp

EKS Operations Handbook

Version 3.2 - Platform Reliability Team

This document is a synthetic test document containing fictional internal standards. It exists to test PDF parsing, chunking, retrieval and citation behaviour of a RAG application.

1. Introduction and Scope

1.1 Purpose of this handbook

This handbook describes how the Platform Reliability team at Acme Corp operates Amazon EKS clusters. It is the single source of truth for cluster standards, upgrade procedures, security controls and incident handling. Engineers who deploy workloads to EKS are expected to read the chapters that apply to them before requesting namespace access.

The handbook covers three clusters. The production cluster is named prod-use1-core, the staging cluster is named staging-use1-core, and the sandbox cluster used for experiments is named dev-euw1-sandbox. Anything not listed here must be approved by the Platform Reliability team before it is created.

1.2 Ownership and change windows

The Platform Reliability team owns all three clusters and is reachable in the #platform-reliability channel. Application teams own their namespaces, their Helm releases and their service-level objectives.

Planned changes to prod-use1-core may only be applied during the standard change window, which is Tuesday and Thursday from 02:00 to 04:00 UTC. Emergency changes outside this window require approval from the on-call incident commander and must be recorded in the change log within one hour.

2. Cluster Architecture

2.1 Kubernetes version policy

Acme Corp follows an n-1 version policy: production runs one minor version behind the newest version supported by Amazon EKS. At the time of writing prod-use1-core runs Kubernetes 1.31 and staging-use1-core runs 1.32. Staging always upgrades first so that add-on and API compatibility problems are found before production is touched.

A minor version must not remain in production for more than 12 months after it reaches the end of standard support. Upgrading to v1.32. Then verify every add-on version against the compatibility matrix before scheduling production.

2.2 Control plane and availability zones

Every cluster uses a private-only API endpoint. Engineers reach the endpoint through the corporate VPN or through the bastion service, never through the public internet. Control plane logging for api, audit and authenticator is enabled on all clusters and shipped to CloudWatch Logs.

Worker nodes are spread across three availability zones: us-east-1a, us-east-1b and us-east-1c for production. Topology spread constraints are required for any Deployment with more than two replicas so that a single zone failure never removes more than one third of capacity.

3. Networking and the VPC CNI

3.1 VPC CNI configuration

Pod networking uses the Amazon VPC CNI add-on. Prefix delegation is enabled on all clusters so that each node can host more pods without exhausting IP addresses in the primary subnets. The environment variable WARM_PREFIX_TARGET is set to 1 and ENABLE_PREFIX_DELEGATION is set to true.

To avoid consuming scarce corporate address space, pods draw addresses from a secondary CIDR, 100.64.0.0/16, attached to the VPC. This setup is called custom networking and is implemented with one ENIConfig object per availability zone.

```
apiVersion: crd.k8s.amazonaws.com/v1alpha1
kind: ENIConfig
metadata:
  name: us-east-1a
spec:
  subnet: subnet-0a1b2c3d4e5f67890
  securityGroups:
    - sg-0123456789abcdef0
```

3.2 Network policies and DNS

Default-deny network policies are applied to every application namespace. Teams add explicit allow rules for the traffic they need. The kube-system.svc.cluster.local domain is reserved for platform services, and application teams must not create services that shadow platform names.

CoreDNS runs with a minimum of 3 replicas in production and uses the NodeLocal DNSCache to reduce latency. If DNS lookups time out under load, increase the CoreDNS replica count before raising any application-level retry limits.

4. Node Groups and Karpenter

4.1 Managed node groups

System workloads such as CoreDNS, the metrics server and the ingress controller run on a managed node group named ng-system. It uses m6i.large instances with a minimum size of 3 and a maximum size of 6, and is tainted so that application pods are not scheduled onto it.

Managed node groups are upgraded by Amazon EKS using a rolling update with a maximum unavailable setting of 1 node, so that the system capacity is never reduced by more than one instance at a time.

4.2 Karpenter NodePools

Application capacity is provisioned by Karpenter. The default NodePool is named general. Its consolidation policy is WhenEmptyOrUnderutilized and nodes expire after 720 hours so that instances are replaced with current AMIs at least every 30 days.

Batch workloads use a separate NodePool named batch-spot that is restricted to Spot capacity. Batch pods must tolerate interruption and must checkpoint their progress at least every 10 minutes.

```
apiVersion: karpenter.sh/v1
kind: NodePool
```

```
metadata:
  name: general
spec:
  template:
    spec:
      requirements:
        - key: karpenter.sh/capacity-type
          operator: In
          values: ["on-demand"]
      expireAfter: 720h
  disruption:
    consolidationPolicy: WhenEmptyOrUnderutilized
    consolidateAfter: 1m
  limits:
    cpu: "400"
```

5. IAM Roles for Service Accounts

5.1 IRSA naming and setup

Workloads obtain AWS permissions through IAM Roles for Service Accounts (IRSA). Each cluster has an OIDC identity provider, and every IAM role used by a pod must trust only that provider and only one specific service account.

Roles follow the naming pattern eks-irsa-<namespace>-<serviceaccount>. For example, the payments API in the payments namespace uses the role eks-irsa-payments-payments-api. Role names that do not follow the pattern are rejected by the admission policy.

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: payments-api
  namespace: payments
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/eks-irsa-payments-payments-api
```

5.2 Migration to EKS Pod Identity

New workloads should use EKS Pod Identity instead of IRSA where the add-on is available. Existing IRSA roles must be migrated before 31 December 2026. After that date the OIDC trust relationships will be removed from the production account.

Teams that need help migrating can book a session with the platform team. Every migrated workload must pass the permission review in which the policy is reduced to the actions actually used over the previous 30 days.

6. Security Controls

6.1 Pod security and secrets

All application namespaces are labeled with the restricted Pod Security Standard in enforce mode. Privileged containers, host networking and hostPath volumes are not allowed. Exceptions require a written risk acceptance signed by the security team and expire after 90 days.

Kubernetes Secrets are encrypted at rest with the KMS key aliased alias/eks-prod-secrets. Teams should prefer the External Secrets Operator to synchronize values from AWS Secrets Manager rather than committing secret manifests to Git.

6.2 Image provenance

Only images from the company registry 123456789012.dkr.ecr.us-east-1.amazonaws.com may run in production. Images must be scanned on push, and an image with a critical vulnerability that has a published fix cannot be deployed.

Images are referenced by digest rather than by mutable tag in production manifests. The admission controller rejects any pod whose image reference ends in the latest tag.

7. Cluster Upgrades

7.1 Upgrade order

Upgrades always follow the same order: first the control plane, then the managed add-ons (VPC CNI, CoreDNS, kube-proxy and the EBS CSI driver), and finally the

nodes. Skipping a step or reordering them is not permitted because add-on versions are only validated against the matching control plane version.

Amazon EKS does not support rolling back a control plane to a previous minor version. If a problem is found after the control plane upgrade, the only options are to fix forward or to restore workloads into a new cluster built at the older version.

```
# 1. Check for deprecated APIs still in use
kubectl get --raw /metrics | grep apiserver_requested_deprecated_apis

# 2. Upgrade the control plane
eksctl upgrade cluster --name staging-use1-core --version 1.32 --approve

# 3. Update the add-ons
eksctl update addon --cluster staging-use1-core --name vpc-cni --force
```

7.2 Pre-upgrade checklist

Before any upgrade the on-call engineer confirms that all Pod Disruption Budgets allow at least one pod to be evicted, that no node is in the NotReady state, and that the latest etcd-independent backups of persistent volumes completed successfully.

The upgrade ticket must contain the output of the deprecated-API check, the list of add-on versions before and after, and a named rollback owner. Tickets missing any of these items are returned to the requester.

8. Observability

8.1 Metrics and logs

Metrics are collected by Prometheus and shown in Grafana. Container Insights in CloudWatch is enabled as a secondary source for node-level data. Metrics are retained for 15 days at full resolution, while application and control plane logs are retained for 30 days.

Every service must expose a /metrics endpoint and include the labels app, team and env. Series without these labels are dropped by the collector and will not appear in dashboards.

8.2 Alert thresholds

The following alerts are defined for every cluster. Thresholds are intentionally conservative and are reviewed once per quarter.

Alert	Condition	Severity
NodeHighCPU	Node CPU above 85% for 10 minutes	SEV3
PodCrashLooping	More than 5 restarts in 15 minutes	SEV2
NodeNotReady	Node NotReady for 5 minutes	SEV2
APIServerErrors	5xx rate above 2% for 5 minutes	SEV1

9. Autoscaling and Availability

9.1 Horizontal Pod Autoscaler defaults

Services that scale horizontally should use the Horizontal Pod Autoscaler with a target CPU utilization of 65% and a minimum of 2 replicas. A minimum of 1 replica is not allowed in production because a single eviction would cause an outage.

Scale-down is stabilized for 300 seconds to avoid flapping. Teams that need faster scale-down must justify the change in the pull request.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: payments-api
  namespace: payments
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: payments-api
  minReplicas: 2
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 65
```

9.2 Disruption budgets

Every Deployment with two or more replicas must define a Pod Disruption Budget with maxUnavailable set to 1. This allows node drains during upgrades to proceed one pod at a time without taking a service down.

Stateful workloads such as databases running in the cluster use a budget of maxUnavailable 0 and are drained manually under the supervision of the owning team.

10. Storage

10.1 EBS and EFS

The default StorageClass is gp3, provided by the EBS CSI driver. Volume expansion is enabled, and the reclaim policy is Delete for the default class. Workloads that must keep data after the claim is removed need the gp3-retain class instead.

Shared read-write volumes use Amazon EFS through the EFS CSI driver. EFS is not suitable for latency-sensitive databases and should be used mainly for shared content and build caches.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  encrypted: "true"
allowVolumeExpansion: true
reclaimPolicy: Delete
```

10.2 Snapshots and recovery

Volume snapshots are taken daily at 01:00 UTC and kept for 7 days. A restore drill is carried out every quarter by restoring a randomly selected snapshot into the staging cluster and verifying application data.

Snapshots are encrypted with the same KMS key as the source volume. Restoring into another account requires the key policy to be shared explicitly.

11. Troubleshooting Guide

11.1 Common pod errors

When a pod does not start, begin with kubectl describe pod and read the Events section at the bottom. Most problems fall into a small number of categories, summarized in the table below.

Symptom	Likely cause	First action
ImagePullBackOff	Wrong tag or missing ECR permission	Check image name and node role
CrashLoopBackOff	Application exits on start	kubectl logs --previous
Pending	No node with enough CPU or memory	Check Karpenter NodePool limits
FailedCreatePodSandBox	No free IP addresses for the pod	Check subnet free IPs and CNI settings

11.2 IP exhaustion and AccessDenied

The event message failed to assign an IP address to container means the node's subnet or prefix pool is exhausted. Confirm that prefix delegation is active, check the number of free addresses in the pod subnets, and verify that the ENIConfig for the zone points to the secondary CIDR.

The error AccessDenied: User is not authorized to perform an action almost always points to a missing or misnamed IRSA role. Check that the service account annotation matches the role ARN and that the role trusts the cluster OIDC provider and the exact service account.

```
kubectl describe pod payments-api-7d9f8c6b5-x2k4p -n payments
kubectl get events -n payments --sort-by=.lastTimestamp
kubectl logs payments-api-7d9f8c6b5-x2k4p -n payments --previous
aws eks describe-cluster --name prod-use1-core --query cluster.status
```

12. Cost Management

12.1 Spot capacity and tagging

Spot instances may save up to 70% compared with on-demand prices and are used for batch and non-critical workloads only. Customer-facing services must never run exclusively on Spot capacity.

All AWS resources created for the clusters must carry three tags: cost-center, owner and env. Untagged resources are reported weekly and removed after 14 days if no owner claims them.

12.2 Budgets and reviews

A monthly budget is defined per cluster. An alert is sent to the owning team when forecasted spend reaches 80% of the budget, and the platform lead is notified at 100%.

Resource requests and limits are reviewed every quarter using the rightsizing report. Teams whose requests exceed 2 times their observed usage are asked to lower them.

13. Incident Response

13.1 Severity levels

Incidents are classified by customer impact. The response targets below apply 24 hours a day for SEV1 and SEV2 incidents.

Severity	Definition	Response target
SEV1	Customer-facing outage or data loss	15 minutes
SEV2	Significant degradation, workaround exists	30 minutes
SEV3	Minor issue, no customer impact	Next business day

13.2 On-call and postmortems

The on-call rotation changes every Monday at 09:00 UTC. The primary and the secondary engineer are listed in the paging tool, and a handover note is required at each rotation change.

A blameless postmortem must be published within 5 business days of every SEV1 or SEV2 incident. It contains the timeline, the root cause, the customer impact and at least one corrective action with an owner and a due date.

14. Appendix

14.1 Command quick reference

The most frequently used commands are listed below.

Task	Command
List nodes	kubectl get nodes -o wide
Update kubeconfig	aws eks update-kubeconfig --name prod-use1-core
Check add-ons	aws eks list-addons --cluster-name prod-use1-core
Drain a node	kubectl drain NODE --ignore-daemonsets --delete-emptydir-data

14.2 Glossary

IRSA stands for IAM Roles for Service Accounts and lets a pod assume an AWS role through the cluster OIDC provider. PDB stands for Pod Disruption Budget. CNI stands for Container Network Interface. NodePool is the Karpenter resource that defines which nodes may be created.

SEV is short for severity. The change window is the only time in which planned production changes are allowed. Prefix delegation assigns /28 address blocks to node network interfaces instead of single addresses.